

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE SOFTWARE

Plan de Pruebas

Sistema de Chat en Tiempo Real

Integrantes:

César Tipán Antón

Erick Cordova

Camillie Ayovi

Mateo Aguilar

Docente:

Ing. Joel Alejandro Barba Salazar, Mg.

Asignatura:

Aplicaciones Distribuidas

Guayaquil, julio de 2026

Índice general

Plan de pruebas	1
1. Estrategia	1
2. Alcance por componente	1
3. Resultados	2
4. Validación de que las pruebas detectan fallos	3
5. Pruebas manuales de aceptación	3
6. Cobertura no cubierta	3
7. Cómo reproducir	4

Plan de pruebas

1. Estrategia

Las pruebas se reparten en tres niveles, cada uno con un propósito distinto:

Nivel	Herramienta	Qué verifica	Cantidad
Unitarias del servidor	pytest	Lógica de presencia y de seguridad, aislada de la red y de la base de datos	46
Unitarias del cliente	Vitest	Estado de los componentes y su representación en pantalla	29
Extremo a extremo	Playwright	Recorridos completos sobre el sistema real, con navegadores y servidor en marcha	9
Total			84

El reparto responde al coste de cada nivel. La lógica de presencia concentra la mayoría de los casos límite del sistema y es barata de probar en aislamiento, de ahí el peso de las pruebas unitarias. Las pruebas de extremo a extremo son lentas y frágiles, y se reservan para lo que ningún otro nivel puede demostrar: que dos navegadores distintos se vean en tiempo real.

2. Alcance por componente

2.1 Servidor — 46 pruebas

ConnectionManager (29 pruebas). Es el componente con más estado y, por tanto, el más propenso a fallos sutiles:

- Registro de la primera conexión de un usuario frente a las siguientes, distinción usada para decidir si se anuncia su entrada.
- Desconexión de un socket concreto y de todos los sockets de un usuario.
- Un usuario con varias pestañas sigue conectado hasta cerrar la última.

- Composición y orden de la lista de conectados: vacía, poblada y tras una desconexión.
- Difusión a todo el canal, con exclusión opcional del emisor.
- Limpieza de sockets muertos durante la difusión, comprobando que el envío continúa hacia el resto de destinatarios.
- Aislamiento entre canales: ni los mensajes ni la presencia se cruzan.
- Liberación de un canal cuando se marcha su último ocupante, para no acumular memoria.

Seguridad (17 pruebas). Hashing de contraseñas y emisión de tokens:

- El hash nunca coincide con la contraseña en claro.
- Dos hashes de la misma contraseña difieren, lo que confirma que el *salt* es aleatorio.
- La verificación acepta la contraseña correcta y rechaza la incorrecta, incluidos los casos de contraseña vacía y de caracteres Unicode.
- El token transporta el identificador y el nombre de usuario, y lleva marca de expiración.
- Un token manipulado, firmado con otra clave o con otro algoritmo, es rechazado.

2.2 Cliente — 29 pruebas

- **Lista de usuarios conectados (11).** Contador de la cabecera con cero, uno y varios usuarios; presentación de cada entrada; inicial y color del avatar derivados del nombre.
- **Disposición del chat (15).** Carga inicial de canales; apertura y cierre del menú lateral por botón, por tecla Escape y al seleccionar un canal; orden correcto de las operaciones al cambiar de canal (desconectar, seleccionar, conectar); cierre de la conexión al abandonar la vista y al cerrar sesión.
- **Componente raíz (1).** Se construye sin errores.

2.3 Extremo a extremo — 9 pruebas

Se ejecutan contra el servidor y la base de datos reales, con dos navegadores independientes cuando el caso lo exige.

Prueba	Qué demuestra
Sincronización de presencia entre sesiones	Al conectarse un usuario, aparece en la lista del otro sin recargar la página
Presencia con varias pestañas	Cerrar una de dos pestañas no marca al usuario como desconectado
Aislamiento de canales	Al cambiar de canal, el usuario desaparece de la lista del canal anterior
Menú lateral en móvil (3)	Apertura y cierre por botón, por Escape y al elegir canal; el botón flotante no tapa el nombre del canal; en escritorio el menú es fijo y el botón no existe
Identidad visual (3)	Ninguna pantalla dibuja iconos con caracteres de texto; todos son SVG

3. Resultados

Ejecución completa sobre la rama de trabajo:

```
backend  python -m pytest tests -q      46 passed
```

```
frontend npm test                29 passed
frontend npx playwright test     9 passed
```

Las 84 pruebas pasan. Las de extremo a extremo requieren el servidor en `localhost:8000` con la base de datos poblada; Playwright levanta por su cuenta el servidor del cliente.

4. Validación de que las pruebas detectan fallos

Una prueba que nunca falla no demuestra nada. Se comprobó que las pruebas de presencia y del menú lateral son sensibles a la ausencia del código que verifican:

- Ejecutadas contra la versión del servidor **sin** el evento `user_list`, las tres pruebas de presencia fallan, mientras que las dos del menú lateral siguen pasando, porque no dependen del servidor.
- Restaurado el CSS con el defecto de solapamiento, la prueba «el botón flotante no tapa el nombre del canal» falla midiendo la superposición real: el borde derecho del botón cae en el píxel 52 sobre un título que empieza en el 24.

5. Pruebas manuales de aceptación

Caso	Procedimiento	Resultado esperado
Registro con datos válidos	Completar el formulario y enviar	Entra al chat con la sesión iniciada
Registro con correo repetido	Registrar dos veces el mismo correo	Aviso «El email ya está registrado»
Inicio de sesión incorrecto	Contraseña equivocada	Aviso «Credenciales incorrectas»
Ruta protegida	Abrir <code>/chat</code> sin sesión	Redirige a <code>/login</code>
Difusión	Dos navegadores en el mismo canal; escribir en uno	El mensaje aparece en el otro al instante
Persistencia	Recargar la página tras enviar mensajes	El historial se recupera
Indicador de escritura	Escribir sin enviar	El otro navegador muestra el aviso; desaparece a los dos segundos
Aislamiento	Dos navegadores en canales distintos	Ninguno ve los mensajes ni la presencia del otro
Cierre de sesión	Pulsar «Salir»	Regresa a <code>/login</code> y desaparece de la lista del otro navegador

6. Cobertura no cubierta

Se declaran de forma explícita las áreas sin prueba automatizada:

- Los puntos REST (`/auth/register`, `/auth/login`, `/channels`) no tienen pruebas de integración propias. Su comportamiento se ejercita de forma indirecta en las pruebas de extremo a extremo, que inician sesión y cargan los canales en cada recorrido.
- Los repositorios no se prueban contra una base de datos real.
- El manejador WebSocket se prueba a través de la interfaz, no como unidad.
- No hay pruebas de carga ni de concurrencia con muchos clientes simultáneos.

7. Cómo reproducir

1. Base de datos y servidor

```
cd backend && source .venv/bin/activate  
uvicorn app.main:app --reload
```

2. Unitarias del servidor

```
python -m pytest tests -q
```

3. Unitarias del cliente

```
cd frontend && npm test
```

*# 4. Extremo a extremo (requiere el servidor en marcha
y los usuarios de prueba creados)*

```
npx playwright test
```